

Outlook Folder Extractor → LLM-Powered Task Prioritization

What the App Produces

The extractor gives you **5 output formats** from your M365 mailbox — each with different strengths for AI reasoning:

Format	LLM suitability	Why
JSON	★★★★★ Best	Structured, easy to parse, rich fields
SQLite	★★★★★ Best	Queryable, incremental, persistent — ideal for an agent
Emails CSV	★★★★★ Good	Flat, universal, simple to feed as context
EML Files	★★★ Limited	Too verbose, lots of MIME overhead
Recipients CSV	★ Not useful	No content — just addresses

Recommendation for AI use: export as **JSON** (one-shot analysis) or **SQLite** (recurring agent).

The Two Approaches — Honest Comparison

Option A: Let an AI assistant (e.g. Bob) read and reason

Outlook Extractor → Export JSON/CSV → Share with Bob → Prioritized tasks in chat

When it works well:

- Ad-hoc, one-off sessions ("what should I focus on this week?")
- You need **zero setup** — just export JSON and share it
- Small mailbox extracts (a few hundred emails fit in context)
- You want conversational back-and-forth reasoning

Limitations:

- Context window size limits (~200K tokens ≈ ~500–800 emails with bodies)
- No memory across sessions — you re-share every time
- Not automated — always requires your manual intervention
- Privacy consideration: emails leave your machine to reach the LLM API

Option B: Local Ollama LLM application

```
Outlook Extractor (SQLite)
├── Local Python/Node App
│   ├── SQLite reader + query filter
│   │   └── Ollama API (localhost:11434)
│   │       └── LLM (e.g. llama3.2, mistral, qwen2.5)
│   │           └── Priority list / Markdown digest
```

When it works well:

- **Recurring, automated** pipeline (daily digest)
- **Privacy-first** — emails never leave your machine
- Large mailboxes — process incrementally via SQLite
- Want a persistent task list / memory
- Want to embed rules ("flag from my manager = high priority")

Limitations:

- Requires setup (Python app + prompt engineering)
- Local LLM quality depends on the model (smaller models reason less well)
- Slower than cloud LLMs for complex reasoning

Recommendation

Decision Guide

```
How often do you need this?
├── Once or twice a week
│   └── Option A – Bob reads JSON. Zero setup, conversational.
├── Daily / automated
│   └── Option B – Local Ollama app
│       ├── Privacy concern? Yes
│       │   └── 100% local with Ollama – non-negotiable choice
│       └── Privacy concern? No + want best reasoning
│           └── Hybrid: local SQLite reader + Bob for final reasoning
```

If you want to start today with zero code → Option A

1. Export your flagged emails or inbox as **JSON** using the extractor.
2. Share the file with your AI assistant (e.g. Bob in this project).
3. Ask: "Prioritize these by urgency, group by project, and suggest what to tackle first."

The assistant can reason over the full JSON, group by sender/thread/subject patterns, detect deadlines in subject lines, and produce a prioritized action list.

If you want a recurring automated pipeline → Option B

The ideal local stack with the tools already available on this machine:

Component	Tool
Email store	emails.sqlite from this extractor
LLM runtime	Ollama (locally installed)
Recommended models	llama3.2:latest, mistral, or qwen2.5:7b
Orchestration	Python + ollama SDK + sqlite3
Output	Markdown digest or HTML dashboard

A local pipeline would:

1. Read from emails.sqlite — incremental, only new messages since the last run.
2. Send structured email summaries to Ollama in batches.
3. Ask the LLM to assign priority (high / medium / low), reason why, and suggest a next action.
4. Output a timestamped output/priorities_TIMESTAMP.md digest.

Best Format for LLM Consumption

When feeding emails to an LLM, structure the payload to reduce token waste:

```
{
  "message_id": "...",
  "sent_datetime": "2025-06-25T09:00:00Z",
  "folder": "Inbox",
  "from_email": "alice@example.com",
  "from_name": "Alice",
  "subject": "Urgent: contract renewal deadline Friday",
  "body_text": "Hi, just a reminder that the contract expires this Friday..."
}
```

Fields to always include for task reasoning:

Field	Why it matters
subject	Contains urgency signals and topic
from_email / from_name	Sender priority (manager, client, etc.)
sent_datetime	Recency and deadline detection
body_text	Content for action extraction
folder	Context (Inbox vs. flagged vs. project folder)

Fields to exclude to save tokens:

- body_html — redundant if body_text is included
- attachments metadata — not useful for prioritization unless filenames hint at urgency
- message_id — only needed if you write results back to the database

Recommended Ollama Models for Email Reasoning

Model	Size	Strengths
llama3.2:latest	3B	Fast, good for summarization
mistral:latest	7B	Strong reasoning, good instruction following
qwen2.5:7b	7B	Excellent multilingual + structured output
llama3.1:8b	8B	Best balance of reasoning quality and speed locally

Practical Starting Point

Best first step: try Option A today.

1. Export your flagged emails as **JSON** (use the *Flagged emails only* toggle in the extractor).
2. Share the file with Bob in this workspace.
3. Validate what the LLM reasoning output looks like before investing in building a full local pipeline.

Once you are satisfied with the output quality and want to automate it, the local Ollama pipeline (Option B) can be built as a natural next step from this same project.